

SIMATS ENGINEERING

ASSESSMENT TOOL 2

NAME:D.SANJANA

REG NO:192471029

COURSE NAME:Database Management Systems

COURSE CODE:CSA0501

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5

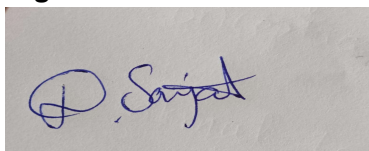
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

I D.SANJANA-192471029 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues

Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. School Management System

Scenario

A school maintains information about students, teachers, classes, and grades. The school administration wants to generate a report showing every student's name along with their class teacher and the grade obtained. This helps teachers and administrators monitor student performance efficiently.

Relations

STUDENT

- StudentID (Primary Key)
- StudentName
- ClassID (Foreign Key)

CLASS

- ClassID (Primary Key)
- ClassName
- TeacherID (Foreign Key)

TEACHER

- TeacherID (Primary Key)
- TeacherName

GRADE

- StudentID (Foreign Key)
- Grade

SQL Query

```
SELECT S.StudentName,  
       C.ClassName,  
       T.TeacherName,  
       G.Grade  
FROM Student S  
JOIN Class C  
ON S.ClassID = C.ClassID  
JOIN Teacher T  
ON C.TeacherID = T.TeacherID  
JOIN Grade G  
ON S.StudentID = G.StudentID;
```

Explanation

- The **Student** table stores student information.

- The **Class** table contains class details and the teacher assigned to each class.
- The **Teacher** table stores teacher names.
- The **Grade** table stores each student's grade.
- The query joins all four tables using their foreign key relationships.
- The final output displays each student's name, class, class teacher, and grade.

Result

StudentName	TeacherName	Grade
Rahul	X-A Priya	A
Sneha	X-B Karthik	B+

2. Hospital Appointment Management

Scenario

A hospital maintains records of patients, doctors, and appointments. The hospital management wants to identify doctors who handled more than **10 appointments during the current month**. This helps evaluate doctors' workloads and resource utilization.

Relational Schema

DOCTOR

- DoctorID (Primary Key)
- DoctorName
- Specialization

PATIENT

- PatientID (Primary Key)
- PatientName
- Age

APPOINTMENT

- AppointmentID (Primary Key)
- PatientID (Foreign Key)
- DoctorID (Foreign Key)
- AppointmentDate

SQL Query

```
SELECT D.DoctorID,
```

```

        D.DoctorName,
        COUNT(A.AppointmentID) AS TotalAppointments
FROM Doctor D
JOIN Appointment A
ON D.DoctorID = A.DoctorID
WHERE MONTH(A.AppointmentDate) = MONTH(CURDATE())
AND YEAR(A.AppointmentDate) = YEAR(CURDATE())
GROUP BY D.DoctorID, D.DoctorName
HAVING COUNT(A.AppointmentID) > 10;

```

Explanation

- The **Doctor** table stores doctor information.
- The **Appointment** table stores appointment records.
- The query joins both tables using **DoctorID**.
- The **WHERE** clause selects appointments only from the current month.
- **COUNT()** calculates the number of appointments handled by each doctor.
- **GROUP BY** groups records according to each doctor.
- **HAVING** filters only those doctors whose appointment count is greater than 1

Result

DoctorID	DoctorName	TotalAppointments
D101	Dr. Kumar	15
D108	Dr. Priya	18

3. E-Commerce System

Scenario

An online shopping website maintains product and order details. The company wants to identify products that have **never been ordered** so that promotional offers or discounts can be provided to improve sales.

Relations

PRODUCT

- ProductID (Primary Key)
- ProductName
- Price

ORDER_DETAILS

- OrderID
- ProductID (Foreign Key)
- Quantity

SQL Query

```
SELECT ProductID, ProductName
FROM Product
WHERE ProductID NOT IN
(
SELECT ProductID
FROM Order_Details
);
```

Explanation

- The **Product** table stores all available products.
- The **Order_Details** table stores products purchased by customers.
- The subquery retrieves ProductIDs that have already been ordered.
- The outer query selects products whose ProductID does not appear in the Order_Details table.
- These products have never been purchased.

Result

ProductID	ProductName
P105	Wireless Mouse
P110	Bluetooth Speaker

4. University Database

Scenario

A university stores student enrollment information. The administration wants to find students who are enrolled in **both DBMS and Operating Systems (OS)** courses.

Relations

STUDENT

- StudentID
- StudentName

COURSE

- CourseID
- CourseName

ENROLLMENT

- StudentID
- CourseID

Relational Algebra

DBMSStudents $\leftarrow \pi$ StudentID

(
 σ CourseName='DBMS'
 (Enrollment $\bowtie$ Course)
)

OSStudents $\leftarrow \pi$ StudentID

(
 σ CourseName='OS'
 (Enrollment $\bowtie$ Course)
)

Result $\leftarrow$ DBMSStudents $\cap$ OSStudents

Explanation

- Join **Enrollment** and **Course** to obtain course names.
- Select students enrolled in DBMS.
- Select students enrolled in OS.
- Use the **Intersection** ($\cap$) operator to retrieve students common to both sets.

Result

StudentID

S101

S108

These students are enrolled in both DBMS and Operating Systems.

5. Payroll Management System

Scenario

A company maintains employee salary information for different departments. The HR department wants to identify employees whose salary is **below the average salary of their department**. These employees will receive a **10% salary increment**. A **view** is created to simplify this process.

Relational Schema

EMPLOYEE

Attribute	Description
EmpID	Employee ID (Primary Key)
EmpName	Employee Name
DepartmentID	Department ID
Salary	Employee Salary

Create View

```
CREATE VIEW LowSalaryEmployees AS
SELECT EmpID,
       EmpName,
       DepartmentID,
       Salary
FROM Employee E
WHERE Salary <
(
  SELECT AVG(Salary)
  FROM Employee E2
  WHERE E.DepartmentID = E2.DepartmentID
);
```

Update Salary by 10%

```
UPDATE Employee
SET Salary = Salary * 1.10
WHERE EmpID IN
(
  SELECT EmpID
```

```
FROM LowSalaryEmployees  
);
```

Explanation

- The **Employee** table stores employee details and salary.
- The **AVG()** function calculates the average salary for each department.
- The **View (LowSalaryEmployees)** stores employees earning less than the department average.
- The **UPDATE** statement increases their salary by **10%**.
- Using a view improves readability and avoids writing the same query repeatedly.

Sample Output

Before Update

EmpID	EmpName	Department	Salary
101	Rahul	HR	25000
102	Priya	HR	35000
103	Karthik	HR	30000

Average Salary = 30000

After Update

EmpID	EmpName	Salary
101	Rahul	27500
102	Priya	35000
103	Karthik	30000

Conclusion

The view identifies employees earning below their department average, and the update query automatically increases their salary by **10%**, making payroll management more efficient.

6. Banking System

Scenario

A bank stores information about customers, their accounts, transaction history, and branch details. The bank manager wants a report showing customer information, account details, transaction history, and branch information using **JOIN operations**.

Relational Schema

CUSTOMER

- CustomerID (Primary Key)
- CustomerName
- Phone

ACCOUNT

- AccountNo (Primary Key)
- CustomerID (Foreign Key)
- BranchID (Foreign Key)
- Balance

TRANSACTION

- TransactionID (Primary Key)
- AccountNo (Foreign Key)
- Amount
- TransactionDate
- TransactionType

BRANCH

- BranchID (Primary Key)
- BranchName
- Location

SQL Query

```
SELECT C.CustomerName,  
       A.AccountNo,  
       A.Balance,  
       T.TransactionID,
```

```

T.TransactionType,
T.Amount,
T.TransactionDate,
B.BranchName,
B.Location
FROM Customer C
JOIN Account A
ON C.CustomerID = A.CustomerID
JOIN Transaction T
ON A.AccountNo = T.AccountNo
JOIN Branch B
ON A.BranchID = B.BranchID;

```

Explanation

- **Customer** stores customer details.
- **Account** stores bank account information.
- **Transaction** stores all deposits and withdrawals.
- **Branch** stores branch information.
- The query joins all four tables using foreign keys.
- The result provides complete banking information for every transaction.

Sample Output

Customer	Account No	Balance	Transaction	Amount	Date	Branch
Rahul	100101	55000	Deposit	10000	05-08-2026	Chennai
Priya	100102	72000	Withdrawal	5000	06-08-2026	Coimbatore

Conclusion

JOIN operations combine related tables to generate a complete banking report without storing duplicate data.

7. Retail Store Sales Report

Scenario

A retail store records daily sales for different product categories. The store manager wants a report showing only categories whose **total daily sales exceed ₹10,000**.

Relational Schema

SALES

- SaleID (Primary Key)
- Category
- Amount
- SaleDate

SQL Query

```
SELECT Category,  
       SUM(Amount) AS TotalSales  
FROM Sales  
WHERE SaleDate = CURDATE()  
GROUP BY Category  
HAVING SUM(Amount) > 10000;
```

Explanation

- The **Sales** table stores daily sales records.
- **SUM()** calculates total sales for each category.
- **GROUP BY** groups records according to category.
- **HAVING** filters categories whose total sales exceed ₹10,000.
- Only qualifying categories appear in the report.

Sample Output

Category	Total Sales
Electronics	25000
Groceries	18000
Furniture	15000

Conclusion

The query summarizes daily sales category-wise and displays only high-performing categories.

8. Library Management System

Scenario

A library maintains records of members, books, genres, and borrowing details. The librarian wants to identify members who have borrowed books from **every available genre** using **Tuple Relational Calculus (TRC)**.

Relations

MEMBER

- MemberID
- MemberName

BOOK

- BookID
- BookName
- Genre

BORROW

- BorrowID
- MemberID
- BookID

Tuple Relational Calculus

{
M.MemberName |

Member(M)

AND

$\forall G$

(

Genre(G)

$\rightarrow$

$\exists B \exists R$

(

Book(B)

AND B.Genre = G.Genre

AND Borrow(R)

AND R.MemberID = M.MemberID

AND R.BookID = B.BookID

)

)

}

Explanation

- **Member** contains library members.
- **Book** stores book details and genres.
- **Borrow** stores borrowing records.
- The universal quantifier ($\forall$) ensures every genre is considered.
- The existential quantifier ($\exists$) checks that the member has borrowed at least one book from each genre.
- Members satisfying this condition are included in the result.

Sample Output

MemberID MemberName

M101 Rahul

M108 Priya

Conclusion

Tuple Relational Calculus retrieves members who have borrowed books from all available genres, demonstrating the use of universal and existential quantifiers.

9. Inventory Management System

Scenario

An inventory management system stores supplier and product details. The company wants to ensure that:

- Product quantity cannot be negative.
- Every product must belong to an existing supplier.

These requirements are enforced using **CHECK** and **FOREIGN KEY** constraints.

Supplier Table

```
CREATE TABLE Supplier
(  
SupplierID INT PRIMARY KEY,  
SupplierName VARCHAR(50)  
);
```

Product Table

```
CREATE TABLE Product
(  
ProductID INT PRIMARY KEY,  
ProductName VARCHAR(50),  
Quantity INT CHECK (Quantity >= 0),  
SupplierID INT,  
FOREIGN KEY (SupplierID)  
REFERENCES Supplier(SupplierID)  
);
```

Demonstrating CHECK Constraint

```
INSERT INTO Product  
VALUES (101,'Laptop',-5,1);
```

Output

ERROR:
CHECK constraint violated.

Demonstrating FOREIGN KEY Constraint

```
INSERT INTO Product  
VALUES (102,'Mouse',20,99);
```


Output

ERROR:

FOREIGN KEY constraint violated.

Explanation

- The **CHECK** constraint ensures that product quantity is always zero or greater.
- The **FOREIGN KEY** constraint ensures that the supplier exists before a product is inserted.
- These constraints improve data accuracy and maintain referential integrity.

Conclusion

Using CHECK and FOREIGN KEY constraints prevents invalid data from being stored and ensures consistency between related tables.

10. HR Management System

Scenario

A company maintains employee attendance and performance records. To improve reporting speed, the HR department creates **materialized views** that store monthly attendance summaries and performance summaries.

Relational Schema

EMPLOYEE

- EmpID (Primary Key)
- EmpName
- DepartmentID

ATTENDANCE

- AttendanceID (Primary Key)
- EmpID (Foreign Key)
- AttendanceDate
- Status

PERFORMANCE

- PerformanceID (Primary Key)
- EmpID (Foreign Key)
- ReviewMonth

- Rating

Materialized View for Monthly Attendance

```
CREATE MATERIALIZED VIEW MonthlyAttendance
AS
SELECT EmpID,
       MONTH(AttendanceDate) AS Month,
       COUNT(*) AS DaysPresent
FROM Attendance
WHERE Status = 'Present'
GROUP BY EmpID,
         MONTH(AttendanceDate);
```

Materialized View for Performance Summary

```
CREATE MATERIALIZED VIEW PerformanceSummary
AS
SELECT EmpID,
       ReviewMonth,
       AVG(Rating) AS AverageRating
FROM Performance
GROUP BY EmpID,
         ReviewMonth;
```

Explanation

- The **Employee** table stores employee information.
- The **Attendance** table records daily attendance.
- The **Performance** table stores employee ratings.
- The **MonthlyAttendance** materialized view stores the total number of days each employee was present every month.
- The **PerformanceSummary** materialized view stores the average performance rating for each employee each month.
- Materialized views store the computed results physically, making reports much faster than recalculating the data every time.

Sample Output

MonthlyAttendance

EmpID	Month	DaysPresent
--------------	--------------	--------------------

101	8	25
-----	---	----

102	8	27
-----	---	----

PerformanceSummary

EmpID	ReviewMonth	AverageRating
--------------	--------------------	----------------------

101	August	4.6
-----	--------	-----

102	August	4.3
-----	--------	-----

Conclusion

Materialized views improve HR reporting by storing precomputed attendance and performance summaries. This reduces query execution time, improves reporting efficiency, and helps management quickly analyze employee performance and attendance trends.